SIMATS ENGINEERING

ASSESSMENT TOOL 1: Case Study

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO4: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I **GUNA P (192424277)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Case Study Rubric

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Clarity	10	Clear, well-structured, and	Organized and understandable	Limited clarity and structure	Poor clarity; difficult to

		easy to understand	presentation		understand

Total Marks Awarded:

Signature of the Course Faculty

Questions:

SET 2

1.) Employee Performance Analysis

Aim

To analyze employee data using Pandas by examining department-wise and designation-wise employee information, studying the relationship between experience, salary, and performance, detecting unusual values, and visualizing the results.

Algorithm

- Start the program.
- Import the required Python libraries such as Pandas, NumPy, and Matplotlib.
- Create or load the employee dataset.
- Display the first few records and examine the structure of the dataset.
- Check for missing values and statistical information.
- Group employees based on Department and calculate average salary and performance.
- Group employees based on Designation.
- Calculate the correlation between Experience, Salary, and Performance Score.
- Identify unusual salary and performance values using the IQR method.
- Create suitable charts such as a department salary bar chart.
- Display the analysis results.
- Stop the program.

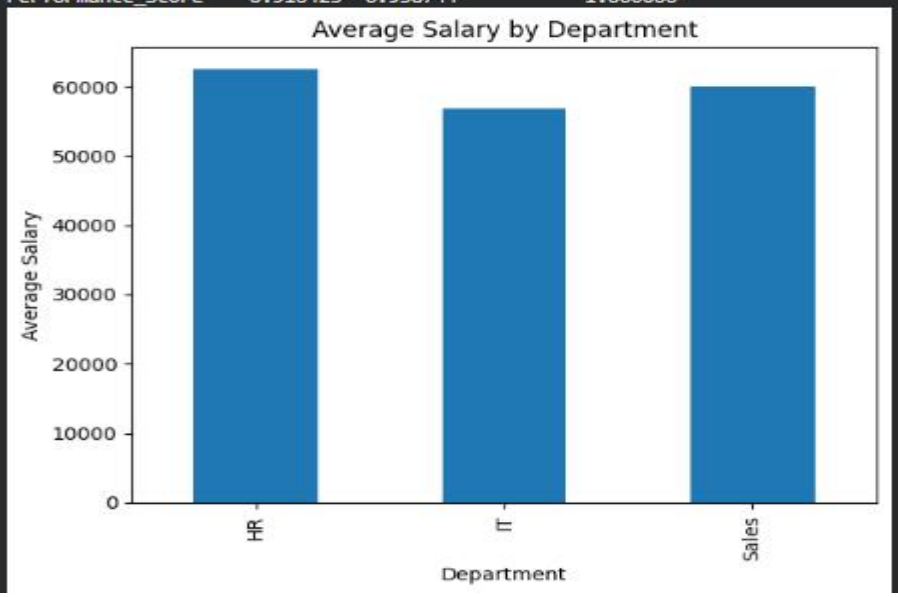
Code

```
 import pandas as pd

import matplotlib.pyplot as plt



# Create sample data

data = {

   "Employee_ID": [101,102,103,104,105,106,107,108],

   "Department": ["IT","HR","IT","Sales","HR","IT","Sales","IT"],

   "Designation":
["Developer","Manager","Tester","Executive","Recruiter","Developer","Manager","Tester"],

   "Salary": [50000,65000,55000,45000,60000,70000,75000,52000],

   "Experience": [2,5,3,1,4,7,8,3],

   "Performance_Score": [75,88,80,65,85,92,90,78]

}
```

```
df = pd.DataFrame(data)

print("EMPLOYEE DATA")
print(df)

# Department analysis
print("\nAVERAGE SALARY BY DEPARTMENT")
print(df.groupby("Department")["Salary"].mean())

print("\nAVERAGE PERFORMANCE BY DEPARTMENT")
print(df.groupby("Department")["Performance_Score"].mean())

# Correlation
print("\nCORRELATION")
print(df[["Experience","Salary","Performance_Score"]].corr())

# Chart
df.groupby("Department")["Salary"].mean().plot(kind="bar")

plt.title("Average Salary by Department")
plt.xlabel("Department")
plt.ylabel("Average Salary")
plt.show()
```

OUTPUT

```
EMPLOYEE DATA
   Employee_ID Department Designation  Salary  Experience  Performance_Score
0          101         IT   Developer   50000           2                 75
1          102         HR     Manager   65000           5                 88
2          103         IT      Tester   55000           3                 80
3          104      Sales   Executive   45000           1                 65
4          105         HR   Recruiter   60000           4                 85
5          106         IT   Developer   70000           7                 92
6          107      Sales     Manager   75000           8                 90
7          108         IT      Tester   52000           3                 78

AVERAGE SALARY BY DEPARTMENT
Department
HR       62500.0
IT       56750.0
Sales    60000.0
Name: Salary, dtype: float64

AVERAGE PERFORMANCE BY DEPARTMENT
Department
HR       86.50
IT       81.25
Sales    77.50
Name: Performance_Score, dtype: float64

CORRELATION
                 Experience    Salary  Performance_Score
Experience         1.000000  0.989827           0.916423
Salary             0.989827  1.000000           0.938744
Performance_Score  0.916423  0.938744           1.000000
```



2.) Banking Transaction Analysis

Aim

To analyze banking transaction data using Pandas by studying account types, branches, transaction amounts, customer transaction frequency, identifying unusually large transactions, and visualizing transaction patterns.

Algorithm

- Start the program.
- Import Pandas, NumPy, and Matplotlib.
- Create or load the banking transaction dataset.
- Display the dataset and examine its structure.
- Check for missing values and statistical information.
- Group transactions based on Account Type.
- Group transactions based on Branch.
- Calculate transaction count, total amount, and average amount.
- Analyze customer transaction frequency.
- Identify unusually large transactions using a threshold or IQR method.
- Create suitable charts to represent transaction amounts.
- Display the results and insights.
- Stop the program.

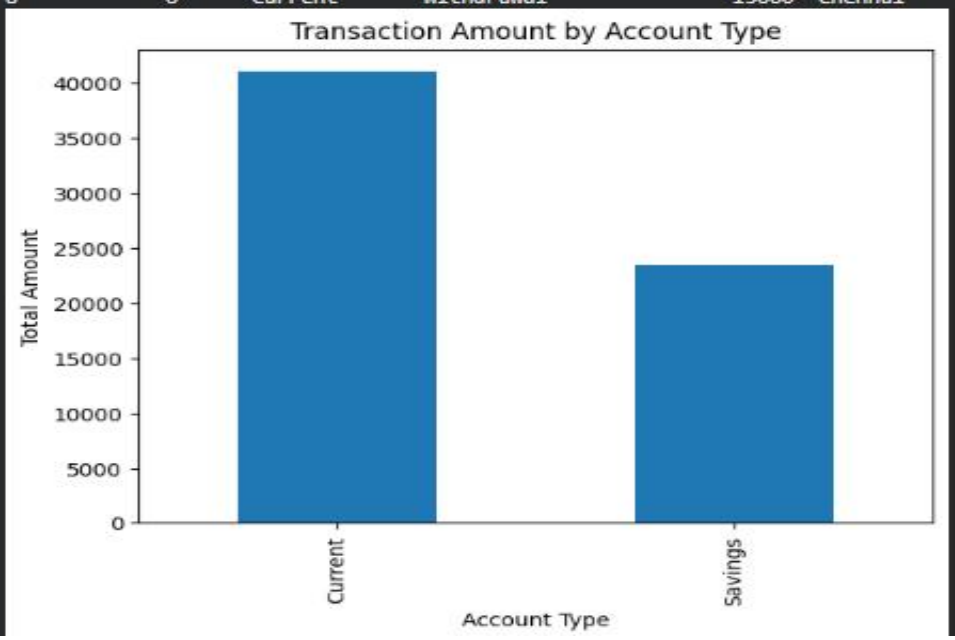
Code

```
import pandas as pd

import matplotlib.pyplot as plt



# Create sample data

data = {

   "Customer_ID": [1,2,3,1,4,2,5,3,6,4],

   "Account_Type": ["Savings","Current","Savings","Savings","Current",

                    "Current","Savings","Savings","Current","Current"],

   "Transaction_Type": ["Deposit","Withdrawal","Deposit","Transfer",

                        "Deposit","Withdrawal","Deposit","Transfer",

                        "Withdrawal","Deposit"],

   "Transaction_Amount": [5000,3000,8000,2500,12000,4000,6000,2000,15000,7000],

   "Branch": ["Chennai","Chennai","Madurai","Chennai","Madurai",
```

```
        "Chennai","Chennai","Madurai","Chennai","Madurai"]
}

df = pd.DataFrame(data)

print("BANKING TRANSACTION DATA")
print(df)

# Account analysis
print("\nTRANSACTIONS BY ACCOUNT TYPE")
print(df.groupby("Account_Type")["Transaction_Amount"].agg(["count","sum","mean"]))

# Branch analysis
print("\nTRANSACTIONS BY BRANCH")
print(df.groupby("Branch")["Transaction_Amount"].agg(["count","sum","mean"]))

# Find large transactions
print("\nLARGE TRANSACTIONS")
print(df[df["Transaction_Amount"] > 10000])

# Chart
df.groupby("Account_Type")["Transaction_Amount"].sum().plot(kind="bar")

plt.title("Transaction Amount by Account Type")
plt.xlabel("Account Type")
plt.ylabel("Total Amount")
plt.show()
```

OUTPUT

```
BANKING TRANSACTION DATA
   Customer_ID Account_Type Transaction_Type  Transaction_Amount   Branch
0            1      Savings          Deposit                5000  Chennai
1            2      Current       Withdrawal                3000  Chennai
2            3      Savings          Deposit                8000  Madurai
3            1      Savings         Transfer                2500  Chennai
4            4      Current          Deposit               12000  Madurai
5            2      Current       Withdrawal                4000  Chennai
6            5      Savings          Deposit                6000  Chennai
7            3      Savings         Transfer                2000  Madurai
8            6      Current       Withdrawal               15000  Chennai
9            4      Current          Deposit                7000  Madurai

TRANSACTIONS BY ACCOUNT TYPE
              count    sum    mean
Account_Type
Current           5  41000  8200.0
Savings           5  23500  4700.0

TRANSACTIONS BY BRANCH
         count    sum         mean
Branch
Chennai      6  35500  5916.666667
Madurai      4  29000  7250.000000

LARGE TRANSACTIONS
   Customer_ID Account_Type Transaction_Type  Transaction_Amount   Branch
4            4      Current          Deposit               12000  Madurai
8            6      Current       Withdrawal               15000  Chennai
```



3.) E-Commerce Product Analysis

Aim

To analyze e-commerce product, sales, and review data using Pandas by integrating multiple datasets, identifying best-selling products, analyzing categories, prices, ratings, and sales, and visualizing the results.

Algorithm

- Start the program.
- Import Pandas, NumPy, and Matplotlib.
- Create or load the product, sales, and review datasets.
- Explore the structure of each dataset.
- Merge the datasets using the common Product_ID.
- Display the combined dataset.
- Group products based on Category.
- Calculate total sales and quantity sold.
- Sort products to identify the best-selling products.
- Calculate the correlation between Price, Sales, and Rating.
- Identify unusually high or low sales values.
- Create suitable charts for category and product sales.
- Display the results and recommendations.
- Stop the program.

Code

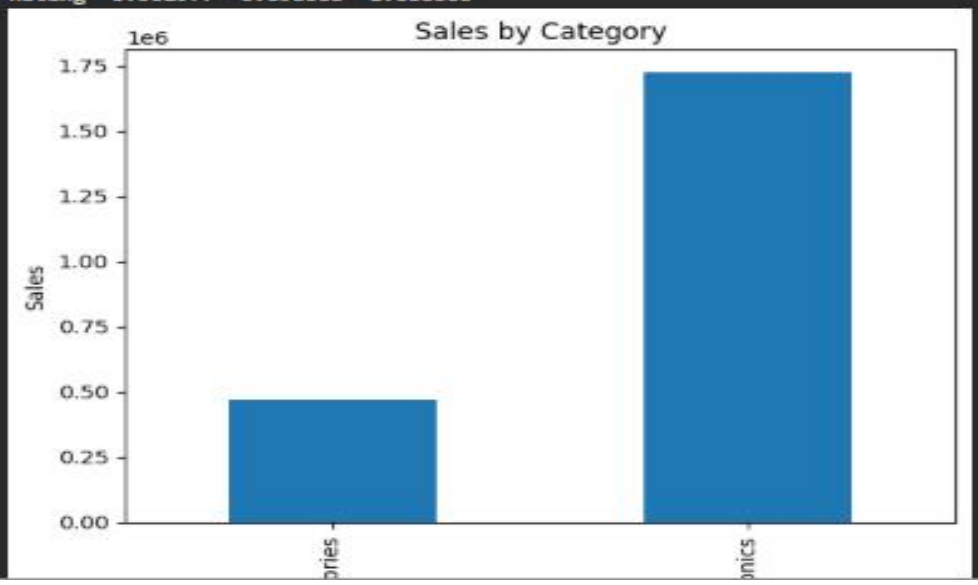
```
import pandas as pd

import matplotlib.pyplot as plt


# Product data

products = pd.DataFrame({

    "Product_ID": [1,2,3,4,5,6],

    "Product": ["Laptop","Mobile","Headphones","Tablet","Keyboard","Watch"],

    "Category": ["Electronics","Electronics","Accessories",

                 "Electronics","Accessories","Accessories"],

    "Price": [60000,30000,3000,25000,1500,5000]

})


# Sales data

sales = pd.DataFrame({
```

```
    "Product_ID": [1,2,3,4,5,6],
    "Quantity": [10,25,50,15,80,40],
    "Sales": [600000,750000,150000,375000,120000,200000]
})

# Review data
reviews = pd.DataFrame({
    "Product_ID": [1,2,3,4,5,6],
    "Rating": [4.5,4.7,4.2,4.4,3.8,4.6]
})

# Merge datasets
df = products.merge(sales, on="Product_ID")
df = df.merge(reviews, on="Product_ID")

print("E-COMMERCE DATA")
print(df)

# Category analysis
print("\nSALES BY CATEGORY")
print(df.groupby("Category")["Sales"].sum())

# Best products
print("\nBEST SELLING PRODUCTS")
print(df.sort_values("Sales", ascending=False)[
    ["Product","Sales","Rating"]
])

# Correlation
print("\nCORRELATION")
print(df[["Price","Sales","Rating"]].corr())

# Chart
df.groupby("Category")["Sales"].sum().plot(kind="bar")
```

```
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()
```

OUTPUT

```
E-COMMERCE DATA
   Product_ID     Product     Category  Price  Quantity   Sales  Rating
0           1      Laptop  Electronics  60000        10  600000     4.5
1           2      Mobile  Electronics  30000        25  750000     4.7
2           3  Headphones  Accessories   3000        50  150000     4.2
3           4      Tablet  Electronics  25000        15  375000     4.4
4           5    Keyboard  Accessories   1500        80  120000     3.8
5           6       Watch  Accessories   5000        40  200000     4.6

SALES BY CATEGORY
Category
Accessories     470000
Electronics    1725000
Name: Sales, dtype: int64

BEST SELLING PRODUCTS
      Product   Sales  Rating
1      Mobile  750000     4.7
0      Laptop  600000     4.5
3      Tablet  375000     4.4
5       Watch  200000     4.6
2  Headphones  150000     4.2
4    Keyboard  120000     3.8

CORRELATION
            Price     Sales    Rating
Price    1.000000  0.812840  0.502977
Sales    0.812840  1.000000  0.698568
Rating   0.502977  0.698568  1.000000
```



4.) Weather Data Analysis

Aim

To analyze weather data using Pandas by studying temperature, humidity, rainfall, and wind speed patterns across different cities and time periods, identifying unusual observations, and visualizing weather trends.

Algorithm

- Start the program.
- Import Pandas, NumPy, and Matplotlib.
- Create or load the weather dataset.
- Display the first few records and examine the dataset structure.
- Check for missing values and statistical information.
- Convert the Date column into a date format.
- Group weather observations based on City.
- Calculate average temperature, humidity, rainfall, and wind speed.
- Analyze weather patterns over time.
- Calculate correlations between temperature, humidity, rainfall, and wind speed.
- Identify unusual temperature and rainfall observations.
- Create temperature and rainfall charts.
- Display the analysis results.
- Stop the program.

Code

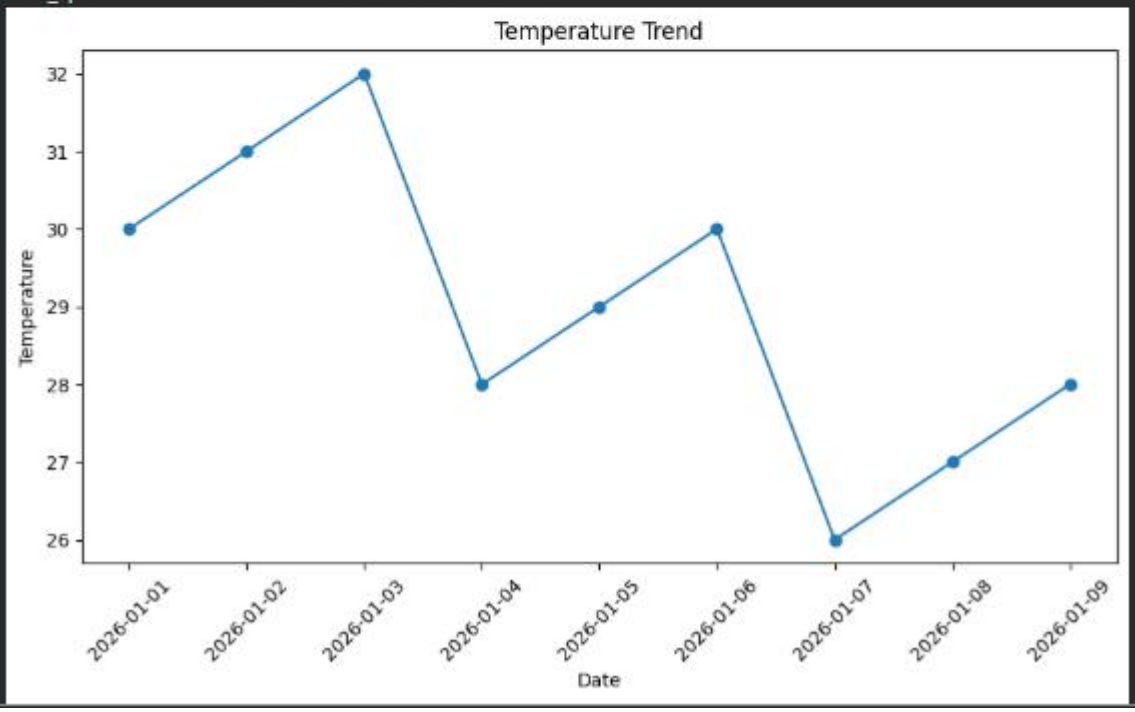
```
import pandas as pd

import matplotlib.pyplot as plt



# Create sample weather data

data = {

   "City": ["Chennai","Chennai","Chennai","Madurai","Madurai","Madurai",

         "Coimbatore","Coimbatore","Coimbatore"],

   "Date": pd.date_range("2026-01-01", periods=9),

   "Temperature": [30,31,32,28,29,30,26,27,28],

   "Humidity": [70,72,75,65,68,70,80,82,78],

   "Rainfall": [10,5,20,2,0,8,15,25,10],

   "Wind_Speed": [12,14,10,8,9,11,15,18,13]

}
```

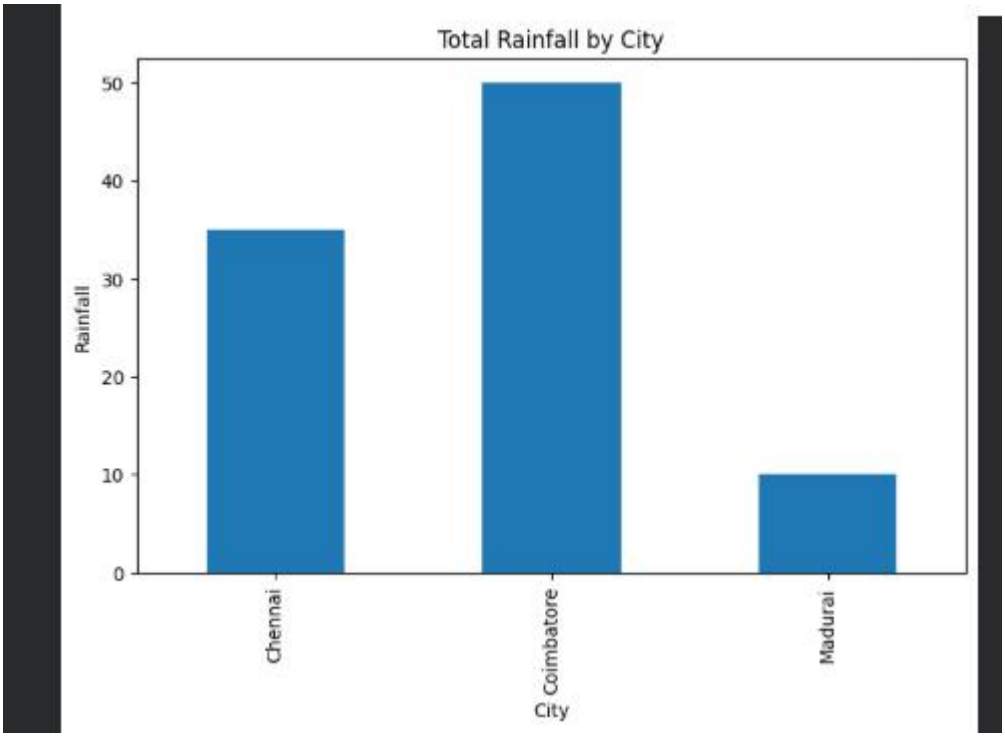
```
df = pd.DataFrame(data)

print("WEATHER DATA")
print(df)

# City analysis
print("\nAVERAGE WEATHER BY CITY")
print(df.groupby("City").agg({
    "Temperature": "mean",
    "Humidity": "mean",
    "Rainfall": "sum",
    "Wind_Speed": "mean"
}))

# Correlation
print("\nCORRELATION")
print(df[[
    "Temperature",
    "Humidity",
    "Rainfall",
    "Wind_Speed"
]].corr())

# Temperature chart
plt.figure(figsize=(10,5))

plt.plot(
```

```
    df["Date"],
    df["Temperature"],
    marker="o"
)

plt.title("Temperature Trend")
plt.xlabel("Date")
plt.ylabel("Temperature")
plt.xticks(rotation=45)
plt.show()

# Rainfall chart
plt.figure(figsize=(8,5))

df.groupby("City")["Rainfall"].sum().plot(kind="bar")

plt.title("Total Rainfall by City")
plt.xlabel("City")
plt.ylabel("Rainfall")
plt.show()
```

OUTPUT

```
WEATHER DATA
         City       Date  Temperature  Humidity  Rainfall  Wind_Speed
0     Chennai 2026-01-01           30        70        10          12
1     Chennai 2026-01-02           31        72         5          14
2     Chennai 2026-01-03           32        75        20          10
3     Madurai 2026-01-04           28        65         2           8
4     Madurai 2026-01-05           29        68         0           9
5     Madurai 2026-01-06           30        70         8          11
6  Coimbatore 2026-01-07           26        80        15          15
7  Coimbatore 2026-01-08           27        82        25          18
8  Coimbatore 2026-01-09           28        78        10          13

AVERAGE WEATHER BY CITY
             Temperature   Humidity  Rainfall  Wind_Speed
City
Chennai             31.0  72.333333        35   12.000000
Coimbatore          27.0  80.000000        50   15.333333
Madurai             29.0  67.666667        10    9.333333

CORRELATION
             Temperature  Humidity  Rainfall  Wind_Speed
Temperature     1.000000 -0.425385 -0.149247   -0.429856
Humidity       -0.425385  1.000000  0.826570    0.847821
Rainfall       -0.149247  0.826570  1.000000    0.665132
Wind_Speed     -0.429856  0.847821  0.665132    1.000000
```




Total Rainfall by City
Rainfall
50
40
30
20
10
0
Chennai
Coimbatore
Madurai
City